MSCPHUB MSCP Disk Server for OpenVMS Nodes

Programmer’s Guide

MSCPHUB-PG-0119

MSCPHUB Version X01-23 *** 25-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

MSCPHUB is free software, distributed under the GNU General Public License, Version 3 or later. It comes with ABSOLUTELY NO WARRANTY.

Contents

Preface **4**
Intended Audience . . . 4
Document Structure . . . 5
Acknowledgments . . . 6
Trademarks . . . 7
Conventions . . . 8

Chapter 1. Concept **9**
1.1 Why the Server Exists . . . 9
1.2 What the Server Is . . . 10
1.3 Decisions and Why . . . 11
1.4 What Is Deliberately Not Done . . . 12
1.5 Why Not DPDK, SPDK, netmap or PF_RING . . . 13
1.6 A Course of Study . . . 14

Chapter 2. Architecture **15**
2.1 The Layers . . . 15
2.2 The Event Loop . . . 17
2.3 The Path of a READ . . . 18

Chapter 3. The Modules **19**
3.1 mscphub.c . . . 19
3.2 mh_lan.c . . . 20
3.3 mh_cc.c . . . 21
3.4 mh_tr.c . . . 22
3.5 mh_ppd.c . . . 23
3.6 mh_scs.c . . . 24
3.7 mh_blk.c . . . 25
3.8 mh_mscp.c . . . 26
3.9 mh_bk.c . . . 27
3.10 mh_utl.c . . . 28
3.11 mh_hlp.c . . . 29
3.12 mh_stat.c . . . 30
3.13 mh_mbus.c . . . 31
3.14 mhmon.c . . . 32
3.15 mh_ods.c . . . 33
3.16 mh_mop.c . . . 34

Chapter 4. Data Structures **35**

Chapter 5. Conventions **36**
5.1 Naming . . . 37
5.2 Formatting . . . 38
5.3 The Module Header . . . 39
5.4 IDENT and REV . . . 40
5.5 The Calling Convention . . . 41

5.6 Diagnostics . . . 42
5.7 Strings and Lengths . . . 43
5.8 The Build Files and the Shell . . . 44
5.9 The Documentation . . . 45

Chapter 6. Making a Change **46**
6.1 Adding a Qualifier . . . 46
6.2 Adding a Message . . . 47
6.3 Adding an MSCP Command . . . 48
6.4 Adding a Counter to MHMON and Modbus . . . 49
6.5 Releasing a Version . . . 50

Chapter 7. The Build System **51**
7.1 Layout . . . 51
7.2 Dependencies . . . 52
7.3 Options . . . 53
7.4 The Checks of the Configuration . . . 54
7.5 The PDF of the Manuals . . . 55
7.6 The Kit . . . 56

Chapter 8. Testing **57**
8.1 What Is There . . . 57
8.2 Rules . . . 59

Appendix A. Known Limitations **60**

Preface

Intended Audience

This manual is for the programmer who is to alter the server: to add a qualifier, a message or an MSCP command, to correct a defect, or to satisfy himself that a change is safe. It assumes C, POSIX and a reading acquaintance with CMake. It does not assume OpenVMS, although the reader who knows it will recognize why much of the design looks as it does.

The *MSCPHUB User's Guide* describes what the server does. This manual describes how it is made, and neither repeats the other.

Document Structure

Part	Contents
Chapter 1	Why the server exists, the decisions it rests upon, why it does not bypass the kernel, and how it may be studied
Chapter 2	Architecture: the layers, the event loop, the path of a command
Chapter 3	The modules, one by one
Chapter 4	The data structures
Chapter 5	The conventions the sources hold to, and why
Chapter 6	How to make the changes that come up most often
Chapter 7	The build system and the kit
Chapter 8	The tests and what they guard
Appendix A	Known limitations

Acknowledgments

The authors thank Claude, the AI model of Anthropic, which went through the protocols, the code, the tests, the measurements and these manuals together with them.

The authors thank their comrades-in-arms of the NaPi Lab community, who paid for the subscription to Claude, so that an engineer of almost pensionable age could carry on this research.

Trademarks

OpenVMS, VMScluster, Alpha, AlphaServer, Integrity, Digital, DEC, HSC, HSJ and HSD are trademarks of Hewlett Packard Enterprise Development LP or of VMS Software, Inc. T4 and TLViz are tools of Hewlett Packard Enterprise. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. Intel and Xeon are trademarks of Intel Corporation. Arm and Cortex are trademarks of Arm Limited. Rockchip is a trademark of Fuzhou Rockchip Electronics Co., Ltd. Raspberry Pi is a trademark of Raspberry Pi Ltd. Modbus is a registered trademark of Schneider Electric, licensed to the Modbus Organization, Inc. Debian is a registered trademark of Software in the Public Interest, Inc. DPDK is a project of The Linux Foundation. PF_RING is a trademark of ntop. netmap is free software of Luigi Rizzo and the University of Pisa. NaPi is the name of the boards of the NaPi Lab team. Claude is a trademark of Anthropic, PBC. All other names of products and companies are trademarks of their respective owners.

MSCPHUB is an independent implementation, made for interoperability. It contains no code of OpenVMS and is not affiliated with, endorsed or supported by any of the owners named here; their names are used only to say what MSCPHUB works with.

Conventions

Line numbers are not quoted: they go stale. A routine is named instead, and every routine carries a `FUNCTIONAL DESCRIPTION` block at its head that says what it does, what it takes and what it returns. Where this manual and that block disagree, the block is right and this manual is to be corrected.

Chapter 1. Concept

1.1 Why the Server Exists

The VMScluster of Digital Equipment Corporation was the first cluster that worked: every node sees every disk, a disk is served by whichever node or controller holds it, and the cluster carries on when a member leaves. The machines that ran it are leaving the machine rooms, and the controllers that served its disks - the HSC, the HSJ, the HSD - left long ago. The systems themselves, and the engineering culture they were made with, should not go the same way.

MSCPHUB is one part of the work to keep that heritage alive and in use. It takes the place of the disk controller: OpenVMS nodes - real or emulated, Alpha, Integrity or x86 - keep their disks on the storage of a Linux computer, and nothing in OpenVMS is changed for it.

The second purpose is the culture itself. The sources are written the way the engineers of DEC wrote theirs: the facility and its condition values, messages in the OpenVMS form, a HELP library, qualifiers, IDENTs and a modification history in every module, a `FUNCTIONAL DESCRIPTION` over every routine, manuals laid out like the manuals of OpenVMS. This is deliberate. It is the way of working that made those systems last for forty years - every decision written down, every condition reported, nothing left for the operator to guess - and a way of working is kept only by being used. A programmer who alters the server is asked to keep it so (Chapter 5).

1.2 What the Server Is

The server is an SCS node with one system application, the disk server `MSCP$DISK`, and without a connection manager. It speaks NISCA - the protocol of PEDRIVER - on the Ethernet, so that an OpenVMS node finds it on the LAN, opens a channel and a virtual circuit to it, connects its disk class driver to it, and sends it MSCP commands. It is never a member of the VMScluster and never votes.

1.3 Decisions and Why

No membership in the cluster. A member would have to run the connection manager, take part in state transitions and the quorum, and could hang the cluster by a fault of its own. A controller that only serves disks needs none of it. The server tells OpenVMS that it is not an OpenVMS system, and OpenVMS then treats it as it treats an HSC.

One process, one protocol thread. The protocol, from the frame up to MSCP, runs in one thread on an EPOLL loop. Nothing is locked, because nothing else touches the state. This is a real constraint on what may be added: anything that blocks the loop stops every node. The disk I/O is therefore never done in the loop; it goes to io_uring, and its completions come back as an event of the loop.

What others read is a copy. Every 100 ms the protocol thread copies its state and counters into a block in shared memory, under a sequence lock: MHMON in another process and the Modbus thread in this one read whole copies of it, and neither ever waits for the protocol thread, nor it for them. The Modbus thread is the only other thread of the server, and it touches nothing but the copy.

The answer to a WRITE means that the data is on the medium. OpenVMS never asks a disk to flush a cache of its own. A server that answers a WRITE from the page cache would lose data that OpenVMS believes is written. The default is therefore a durable write for every WRITE; `/UNSAFE` exists for tests and says so when it starts.

Never send what can crash a node. A malformed answer of a disk server can make an OpenVMS node crash. The server checks its own answers before they leave and holds back one that would be wrong (`MSCPERR`); the strictness of the block transfers comes from the same rule.

Silence before speech. The server listens before it sends its first frame, and stops if its name or system ID is already on the LAN: a clash of identity with a node of the cluster can make that node leave the cluster.

Condition values, not error codes. Every routine returns an OpenVMS condition value; the low bit is the success indicator. A failure is told apart by its identification, never by its text.

Someone else's library. Logging, messages, the parsing of the command line, counted strings and the FAO engine come from the StarLet package; the help library comes from the HELP package. The server carries no general-purpose code of its own.

1.4 What Is Deliberately Not Done

- **No membership, no votes, no connection manager** (Section 1.3).
- **No cache of data in the server.** The data of a READ comes from the Linux disk, and the page cache of Linux is not relied upon for a WRITE. The one exception is the cache of the sectors last written on a disk of 4096-octet sectors, which only spares the reads of a read-modify-write.
- **No threads in the protocol.** The backend is io_uring, not a pool of threads; the one other thread serves Modbus out of the published copy.
- **No fallback for old kernels.** io_uring and the other interfaces of the kernels of the last five years are required; there is no second path for an older kernel.

1.5 Why Not DPDK, SPDK, netmap or PF_RING

Taking the network and the disks away from the kernel is a craft the authors practise - they write software that moves traffic with DPDK - and MSCPHUB could be given it. The LAN lives in one module, `mh_lan.c`, behind four routines; a port of DPDK, netmap, PF_RING or AF_XDP would be a module beside it, chosen at configuration time. The disks live in `mh_bk.c`, where SPDK would be a second backend. It is not done, because here it would buy nothing worth its price:

- **The server is not where the time goes.** It spends one or two microseconds on a command; the rest is the disk of the Linux computer and the node at the other end (*MSCPHUB User's Guide*, Appendix C.4). Bypassing the kernel removes the kernel's part of the network: some microseconds of hundreds.
- **A durable WRITE waits for the medium.** SPDK sends the same flush to the same SSD and does not make it faster. Only an SSD with power-loss protection does.
- **SPDK takes the whole NVMe controller from the kernel:** no file system, no partitions, no LVM - the disks of MSCPHUB would no longer be images that `cp` and `dd` can copy, and the root file system of a small computer is often on that very controller.
- **DPDK, netmap and PF_RING take the network adapter from the kernel**, or need a driver of their own; to be fast they keep a CPU core polling at full load, and want huge pages. A small board has none of that to spare, and its adapter no such driver.
- **A node on the same computer gains nothing.** An emulated OpenVMS node on a tap device in a bridge sends its frames through the kernel whatever the server does.
- **Every one is a second path through the code**, to build, test and keep, for a gain inside the spread of the measurements.

When it would pay: many fast nodes on 10 or 25 Gb Ethernet, hundreds of thousands of frames a second. Then the first step is more protocol threads; the next is AF_XDP - the adapter stays with the kernel, and an XDP filter passes only the frames of NISCA, type 60-07; and DPDK, netmap or PF_RING after that, if AF_XDP falls short.

1.6 A Course of Study

The server is meant to be read. It is small, it is one protocol from the Ethernet frame up to the disk, and every decision in it is written down, here and in the sources. That makes it a fair course for students of computer and network engineering: a real cluster protocol, a real disk server, a real event loop, and the conventions of an engineering culture that built systems to last. And the engineer who knew VMS clusters in their day is welcome too: the names are the old ones, and so, mostly, are the ideas - some of them, perhaps, the reader remembers better than this manual tells.

Chapter 2. Architecture

2.1 The Layers

Bottom up:

Module	Layer	What it does
`mh_lan.c`	LAN	The AF_PACKET socket: frames of type 60-07 in and out, the multicast of the group
`mh_cc.c`	Channel control	HELLO, the table of the nodes seen, the channel to each node
`mh_tr.c`	Transport	The virtual circuit: sequence numbers, acknowledgement, window, retransmission
`mh_ppd.c`	Path	The START/STACK/ACK handshake of SCS, HOSTSHUT
`mh_scs.c`	SCS	Connections, credits, the directory service `SCS$DIRECTORY`
`mh_blk.c`	Block transfers	SEND_DATA for a READ, REQUEST_DATA for a WRITE
`mh_mscp.c`	MSCP server	The units, the hosts, the commands and their order
`mh_bk.c`	Backend	The disk I/O of the units, by io_uring
`mh_ods.c`	Files-11	A reader of the volume of a unit, ODS-2 and ODS-5, read only: a file by its path
`mh_mop.c`	MOP	The MOP server of the Alpha satellites: their bootstrap and parameters, from their disk
`mh_utl.c`	Services	The message catalogue, the OpenVMS time, conversions
`mh_hlp.c`	Help	`/HELP`, out of the help library
`mh_stat.c`	Statistics	The block of state and counters in shared memory, and its reader
`mh_mbus.c`	Modbus	The thread that serves the copy over Modbus/TCP

Module	Layer	What it does
`mscphub.c`	Head module	The parameters, the start-up checks, the event loop
`mhmon.c`	MHMON	The monitor, an image of its own: the screens and the T4 files

The definitions they share are in `include/mh_def.h`; the formats of NISCA and of MSCP are in `include/nisca.h` and `include/mscp.h`; the layout of the statistics block, the interface between the server and MHMON, in `include/mh_stat.h`.

2.2 The Event Loop

`s_mh$loop` in `mscphub.c` waits on EPOLL for four sources:

Source	Event
The LAN socket	Frames: drained, each given to `mh$cc_input`; then the acknowledgements owed go (`mh$tr_flush`)
A TIMERFD of 20 ms	The tick of the protocol: `mh$cc_tick` (HELLO, timers of the channels and circuits), `mh$bk_tick` (stuck requests), `mh$stat_tick` (the queues; the copy every fifth tick)
The EVENTFD of io_uring	Disk completions: `mh$bk_reap`
The two MOP sockets, with /SATELLITE*n*	Frames of MOP V3 and V4: each given to `mh$mop_input`; on the tick, `mh$mop_tick` sends again what is unanswered
A SIGNALFD	SIGINT and SIGTERM stop the server; SIGUSR1 writes the state

A routine of any layer that returns a condition of severity F stops the loop; the server then says goodbye to the nodes (`mh$cc_shutdown`), writes its counters and exits.

2.3 The Path of a READ

1. A frame comes in. `mh$cc_input` checks the DX header and the group, and hands the TR packet of an open channel to the transport.
2. The transport (`mh_tr.c`) takes the packet in order, frees what its acknowledgement covers, and gives the message to the path and to SCS.
3. SCS (`mh_scs.c`) finds the connection, takes the credit, and gives the application message to the MSCP server.
4. The MSCP server (`mh_mscp.c`) checks the command - length, opcode, flags, modifier, unit - and makes it a command in progress (`MH$HRB`). It keeps the order of the unit: a sequential command waits for the unit to be idle; READ, WRITE, ERASE and COMPARE run side by side.
5. The backend (`mh_bk.c`) reads the disk into a data buffer by io_uring. The loop goes on serving.
6. The completion comes on the EVENTFD; `mh$bk_reap` gives the request back to the MSCP server.
7. The block transfer (`mh_blk.c`) sends the data to the buffer of the node in chunks, as the window of the transport opens. The node confirms the last chunk.
8. The MSCP server sends the end message. The transport carries it; the acknowledgement of the node's packets rides on it.

A WRITE goes the other way: the server asks for the data (REQUEST_DATA), the node's PEDRIVER sends it in chunks, the backend writes it durably, and only then does the end message go.

Chapter 3. The Modules

3.1 mscphub.c

The head module. It holds the table of the qualifiers (`s_opts`), checks the configuration (`s_mh$check_cfg`), opens the layers in order and runs the loop. Its `__IDENT__` is the version of the product: CMake reads it from here, `STARTUP` and `UPTIME` report it. `s_mh$stats` writes the messages of `SIGUSR1`.

3.2 mh_lan.c

The socket is `SOCK_RAW` of `AF_PACKET`, bound to the interface and to type 60-07. The Ethernet header and the length word of the DEC padding are built here; the frame is padded to the Ethernet minimum. Frames the server has sent itself come back to such a socket and are dropped and counted. `a_port->xmit`, when set, takes a frame in place of the socket: the tests use it.

With /SATELLITE*n* two more sockets are opened (`mh$lan_mop_open`): one of type 60-01 for MOP V3, one of 802.2 for MOP V4, whose SNAP 08-00-2B-60-01 `mh_mop.c` checks. Both join the multicast of MOP, AB-00-00-01-00-00. `mh$lan_mop_send` takes a frame made whole by `mh_mop.c`.

3.3 mh_cc.c

The DX header, the CC messages, HELLO, the table of the nodes seen on the LAN, and the state machine of the channel to each node. Before its first HELLO the server is silent for `/LISTEN` seconds and learns the nodes; a node that uses its system ID or its name stops it (`IDCLASH`, `NAMECLASH`), and the check stays on for the life of the server. A CC message with a wrong password hash is counted and does not move the channel (`AUTHFAIL`). With `/PASSIVE` the state machine does not run.

A SOLICIT_SRV is the loader of a satellite asking for the server of its system disk. When the disk is one of the units, writable (`mh$mscp_check_service`), the channel goes to SOLICIT_DLY and, 0 to 0.5 seconds later, the server sends VERF; the loader makes a channel of nothing else. Such a channel is marked `boot`: it is kept 30 seconds without a word instead of 8, and when it opens the server starts the path itself. The CCSTART of the runtime PEDRIVER of the satellite restarts the channel as an ordinary one. `SOLICIT` tells the operator once a boot.

An entry of the table is an adapter of a node, and the channel to it. A node of two adapters on the LAN of the server - a satellite of a DE500 and an 82540EM - has two entries of the same system ID and name: the first one seen is the owner (`$MHOWNER`), and the circuit, the connections and the units ONLINE are of it; the other points at it. Each entry has its own handshake, channel sequence numbers and buffer. A TR packet is taken by the circuit of the owner whichever open channel of the node it comes by, its channel sequence number checked against that channel. `mh$cc_chan` picks the channel a packet of the circuit goes by: of the open ones whose buffer takes it, the one of the largest buffer, the one used last while it is as good. The circuit closes with the last channel of the node, not before; a served SOLICIT_SRV closes the other channels of the node, since a node that boots has no runtime channel left. An owner whose circuit runs over another adapter is never taken for a new node when the table is full.

3.4 mh_tr.c

The transport of the virtual circuit: sequence numbers in longwords, a cumulative acknowledgement, a window of at most 16 packets, a cache of the packets that come out of order, and retransmission - 500 ms, doubled up to 3 s; 30 timeouts in a row fail the circuit. An acknowledgement owed goes on the next packet of the server, or alone as `/ACK` says. The window is filled from the queue of messages first, then with the chunks of the block transfer at the head of the circuit, which are made only when there is room.

Every packet with an acknowledgement carries FLAG_EXT with LAST_ACCEPTED equal to it, as PEDRIVER sends them. A VMS node takes the time from its packet to that acknowledgement as the delay of the channel (`SCACP SHOW CHANNEL`); without LAST_ACCEPTED it measures the channel by probes only.

The packets of a circuit - 16 in the window, 64 in the queue, 32 in the receive cache - have their buffers in one pool, allocated with the circuit and sized by the MTU of the port: 1426 octets of data a chunk on an Ethernet, 8926 with jumbo frames of an MTU of 9000. A slot owns its buffer: a queued message is copied into the slot of the window, and the slots an ACK frees hand their buffers to the tail that the move of the window leaves behind.

A packet goes by the channel `mh$cc_chan` picks, with the address and the channel sequence number of that channel, and a chunk of a block transfer is as large as the best open channel of the node takes (`mh$cc_bufsz`). When the channel of jumbo frames closes with chunks of 8100 octets in the window, no open channel takes them: they are not sent to the other adapter, the circuit is failed on the next tick (`NOFIT`), and the node starts it again in the size of the channels it has.

3.5 mh_ppd.c

The path of SCS to a node over its circuit: the server answers the START of the node with STACK, takes the system block of the node out of the START, and opens its transport. HOSTSHUT closes it.

The loader of a satellite never sends START: it waits for the START of its server. On a channel marked `boot` the server sends START every second until the STACK comes (state ST_SENT), and answers the STACK with ACK. START and STACK are built by one routine, `s_mh$ppd_sb`, so they say the same about the server: the satellite makes the system block of its disk server from the START, and its runtime SCS meets that block again in the STACK.

3.6 mh_scs.c

The acceptor half of SCS: the server accepts connections and never asks for one. Two listeners: `SCS$DIRECTORY` and, when disks are served, `MSCP$DISK`. Credits are spent on every application message sent and given back on the next. The directory answers lookups: `MSCP$DISK` is present when disks are served, the other well-known names are answered “not present here”.

3.7 mh_blk.c

The block transfers. SEND_DATA (READ): the chunks go as the window opens, one transfer at a time per circuit; the end message goes only after the node confirms the last chunk. REQUEST_DATA (WRITE, COMPARE): one request for all the data; the chunks come in. A chunk never goes past the byte count of its command; a chunk that comes for nothing of the server’s is dropped and reported once (`BLKERR`).

3.8 mh_mscp.c

The MSCP server: units, hosts (one per connection of a disk class driver), the check of a command, the order of the commands on a unit, and the commands. Immediate commands (ABORT, GET COMMAND STATUS, GET UNIT STATUS, SET CONTROLLER CHARACTERISTICS and the like) are answered at once; the others become an `MH$HRB`. Data buffers come from a pool of 16 MB; a command that finds it empty waits. An end message that would crash the node is held back (`MSCPERR`) and the connection ended. The counters of `UNITSTATS`, `BKSTATS` and `LATSTATS` are kept here.

3.9 mh_bk.c

The backend. The MSCP server hands it a request (`MH$BKREQ`); it goes to io_uring as one step or a few. A device of sectors larger than 512 octets takes a request that is not on whole sectors through a bounce buffer: a READ reads the whole sectors and copies the middle out, a WRITE reads the first and the last sector (or finds them in the cache of the 128 sectors last written), puts its data between them and writes them all. Every WRITE of such a device holds its range of sectors; one that meets a range held waits, in order. A WRITE is made durable as `/SYNC` says. ERASE writes zeros to a file up to 1 MB and uses `fallocate` beyond. A request in the backend longer than 15 seconds is reported (`IOSTUCK`) and cancelled.

A file is opened twice: once with `O_DIRECT` for the I/O of whole blocks, once through the page cache for the rare count that is not a multiple of 512. A file system that refuses `O_DIRECT` leaves the file on the page cache (`NODIRECT`).

3.10 mh_utl.c

The message catalogue (`s_msgtab`) and its registration, the OpenVMS time and its zone, conversions of hexadecimal and MAC address notations.

3.11 mh_hlp.c

`/HELP`, answered out of the help library by the HELP package. The library is looked for in `MSCPHUB_HELPLIB`, in the installed tree, beside the image, and where the build put it. Built without the HELP package, the server answers `/HELP` by a short summary.

3.12 mh_stat.c

The statistics block, `/dev/shm/mscphub.`*node*, mode 0640, made when the server starts and removed when it stops. `mh$stat_tick` adds the queue of every unit to its sum on every tick, and every fifth tick copies the state in: SEQ is made odd, the copy is made, SEQ is made even. `mh$stat_read` is the reader of both MHMON and Modbus: it copies the block between two reads of SEQ and takes the copy only when SEQ was even and did not change; it refuses a block of another MAGIC, VERSION or SIZE. A block that cannot be made is `NOSTAT`, a warning.

3.13 mh_mbus.c

The Modbus/TCP server: `mh$mbus_start` binds and listens in the protocol thread, so that a wrong `/MODBUS` is said at start-up, then starts the thread with every signal blocked. The thread polls the listening socket, at most 8 clients and an EVENTFD that stops it. `mh$mbus_answer` takes a whole copy of the block for every request, lays it out as the registers of the map (User's Guide, Appendix D) and answers 03, 04 and 2B/0E; anything else is an exception.

3.14 mhmon.c

MHMON, an image of its own linked with the same library. It maps the block read only, takes a copy every `/INTERVAL` on the boundaries of the clock, and works out the rates from the differences of two copies over the time the server stamped on them. A copy of another PID or start time starts the statistics afresh; a PID that is gone unmaps the block, and MHMON maps it again when the server is back. The screens are made with FAO into a buffer and written at once. The T4 file is written row by row; its columns are fixed when it is begun, and a node or a disk is found by its name.

3.15 mh_ods.c

A reader of the Files-11 volume of a unit, ODS-2 and ODS-5, read only. `mh$ods_open` finds the home block (LBN 1, or the first block that names itself), and the header of INDEXF.SYS after the index file bitmap; the map of INDEXF.SYS gives the block of any other header. `mh$ods_lookup` walks a path from the MFD, directory by directory, the highest version of a name; an alias such as SYSCOMMON of a root is followed like any other entry. `mh$ods_read` reads blocks of a file by its map, extension headers included. A header is taken only with its checksum right, of structure level 2 or 5, and of the FID asked for, sequence number and all.

The volume is read through the descriptor of the unit, which may be O_DIRECT: every read is of whole pages into a buffer of the reader. The offsets are 64-bit; `$MIN` of STARLET takes `int` and is not used for them. The reader takes no lock: what it reads is what a node does not write while a satellite boots.

3.16 mh_mop.c

The MOP server of the satellites of /SATELLITE*n*. A satellite has a state of its own (`MH$MOPSAT`): only the adapters named are answered, each load goes its own way. A multicast REQUEST PROGRAM makes the load ready - the first image section of `[`*root*`.SYSCOMMON.SYSEXE]APB.EXE`, read by `mh_ods.c` past the header of HDRBLKCNT blocks, and the parameters of the satellite - and only then is ASSISTANCE VOLUNTEER sent: a server that cannot load does not offer to. A REQUEST PROGRAM to the server starts the load with MEMORY LOAD 0. A REQUEST MEMORY LOAD of the number sent asks for it again, of the next one moves on; another ends the load.

After the image comes a MEMORY LOAD of the parameters at the address the image ends at: the bootstrap looks for them right after itself. They are items of a word of code and a word of length: the root, the name of the server, the name of the service of the unit and its UNIT_ID, the system ID of the server, then SCSNODE, SCSSYSTEMID, NISCS_PORT_SERV and NISCS_CONV_BOOT of `[`*root*`.SYSEXE]ALPHAVMSSYS.PAR` in the order they are in that file, then the group and the password. Then the PARAMETER LOAD WITH TRANSFER ADDRESS. The load of an OpenVMS boot server, recorded on the LAN, is the same octet for octet but for the name and the system ID of the server.

Chapter 4. Data Structures

All are declared in `include/mh_def.h`.

Structure	What it is
`MH$CFG`	The configuration: the values of the qualifiers
`MH$PORT`	The LAN port and everything below it: the socket, the table of the nodes, the counters
`MH$NODE`	An adapter of a node seen on the LAN: its identity, the channel to it; the owner of the node, the circuit
`MH$VC`	The virtual circuit to a node: the numbers, the window, the queues, the connections
`MH$TRPKT`	A packet in the transmit queue of a circuit
`MH$CDT`	An SCS connection: its state, its credits, its queue
`MH$APPLMSG`	An application message waiting for a credit
`MH$UNIT`	A unit of the MSCP server: its name, size, state and counters
`MH$HRB`	A command in progress
`MH$BKDEV`	The backend side of a unit: the descriptors, the sector, the cache of written sectors
`MH$BKREQ`	A request to the backend
`MH$LAT`	The time counters of one operation of a unit
`MH$STAT`	The statistics block: who the server is, its counters, its nodes and units (`mh_stat.h`)
`MH$STNODE`, `MH$STUNIT`	A node and a unit in the block
`MH$MOPSAT`	A satellite of /SATELLITE*n*: its adapter, unit and root, and the state of its load
`MH$ODS`, `MH$ODSFILE`, `MH$ODSEXT`	A Files-11 volume being read, a file of it, an extent of the file

The ownership is simple, because there is one thread: a structure belongs to the layer that declares its state, and a lower layer never frees what an upper one holds. A command in progress (`MH$HRB`) outlives its connection when the disk still has a request of it; it is freed when the backend gives the request back.

`MH$STAT` is an interface between two images, and is laid out as one: every field has its size in its type, there are no pointers, and `MH$K_STAT_VERSION` is raised by any change of the layout. MHMON refuses a block of another version (`STATVERS`).

Chapter 5. Conventions

The sources hold to the OpenVMS style throughout. This is not decoration: a reader who knows the convention can tell the lifetime and the origin of a name at sight, and the next programmer reads the code the way the last one wrote it. A change that does not hold to these rules is not finished.

5.1 Naming

Form	Meaning
`l_name`	Local variable
`s_name`	Static: a variable, or a routine not exported from its module
`a_name`	Parameter of a routine
`g_name`	Global variable (the server has none)
`m_name`	Bit mask
`v_name`	Bit field in a structure
`mh$module_name`	Exported routine: the facility, the dollar, the module, the name
`s_mh$module_name`	Static routine
`MH$K_NAME`	Constant
`MH$_NAME`	Condition value
`MH$NAME`	Type of a structure
`$NAME`	Macro

The dollar separates the words of a name, as in OpenVMS: `MH$K_BK_CACHE`, `$MHMSG`. A structure is declared with the tag carrying the suffix `_t` and the type name carrying the dollar; the brace stands at column zero and the members are indented:

```
typedef struct mh_lat_t {
    uint64_t    n;          /* Commands                */
    uint64_t    sum, max;       /* From the command to the END          */
} MH$LAT;
```

Ordered constants are an `enum`. Integers carry their size in their type: `uint32_t`, `uint16_t`; values on the wire are put and taken by `nisca$put16`, `nisca$get32` and their kin, never by casting a pointer.

5.2 Formatting

- **Whitesmiths.** The brace of a block stands on its own line at the indentation of the body it opens. The braces of a routine body stand at column zero.
- **Tabs only**, of width eight. No spaces for indentation.
- **Text to column 192**, not 80.
- **Declarations at the head of a routine**, without indentation, grouped by type.
- **A single statement under `if` takes no braces.**
- **A compound condition is parenthesized part by part**, so that nothing depends on the precedence of operators: `if ( (a_len > MH$K_MAX) || !a_buf )`.
- **A loop counter is declared in the `for`**: `for ( int i = 0; i < n; i++ )`.

A routine is laid out so:

```
/*
**++
**  FUNCTIONAL DESCRIPTION:
**
**  Close the port.  The memberships go with the socket.
**
**  FORMAL PARAMETERS:
**
**  a_port      The port context
**
**  RETURN VALUE:
**  STS$K_SUCCESS.
**--
*/
int mh$lan_close    (
        MH$PORT *   a_port
            )
{
    if ( a_port->sd >= 0 )
        close(a_port->sd);

    a_port->sd = -1;

    return  STS$K_SUCCESS;
}
```

The block over a routine says what it does, its `FORMAL PARAMETERS`, its `IMPLICIT INPUTS/OUTPUTS` where it has them, and its `RETURN VALUE`. Comments are detailed where the code is not obvious and absent where it is: half a screen over a constant or a routine of one line is noise.

5.3 The Module Header

Every source file begins with its identification and its history:

```
#define __MODULE__  "MHLAN"
#define __IDENT__   "X01-10"
#define __REV__     "1.10.0"

/*
**++
**
**  FACILITY:   MSCPHUB - MSCP disk server for OpenVMS nodes, over NISCA
**
**  MODULE: mh_lan.c
**
**  ABSTRACT:   ...
**
**  DESCRIPTION: ...
**
**  AUTHOR: StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)
**
**  CREATION DATE:  17-OCT-2026
**
**  MODIFICATION HISTORY:
**
**      20-OCT-2026 RRL X01-10 : Initial public version.
**
**--
*/
```

The author is always “StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)”, and every line of the history carries `RRL` between the date and the IDENT.

5.4 IDENT and REV

`__IDENT__` is the version in the OpenVMS manner: `X.`*aa*`-`*bb*, and `X.`*aa*`-`*bb*`ECO`*cc* for a small correction of a fault, which does not raise *bb*; *cc* is always of two digits. `__REV__` is the same version in the Unix manner, derived from it part for part: `X.01-24` gives `1.24.0`, `X.01-24ECO03` gives `1.24.3`. The modules not changed since X01-24ECO2 keep the older form without the dot (`X01-11ECO1` gives `1.11.1`); CMake takes both. The two are kept together with care; CMake refuses to configure a tree in which a module's `__REV__` does not follow its `__IDENT__`. The tag of a release is the IDENT of the product itself: `X.01-24ECO03`.

A module carries the IDENT of the last version that changed it. The IDENT of the product is that of the head module, `mscphub.c`, and nowhere else is it taken from.

5.5 The Calling Convention

A routine returns a condition value and delivers its result through an output parameter. A condition is tested on its low bit:

```
if ( !(1 & mh$lan_open(&s_port, $ASCPTR(&s_cfg.ifname))) )
    return  1;
```

`$MHMSG` returns the condition it was given, which is what lets a message and a return fit into one statement.

5.6 Diagnostics

- **What the operator is to read** goes through `$MHMSG`, a condition of the catalogue in `mh_utl.c`, in the OpenVMS form: `%MSCPHUB-F-IDCLASH, ...`. The text is in the catalogue, not at the point of the call.
- **What the programmer is to read** goes through `$LOG_FAO` with an explicit `STS$K_*` severity.
- No message, screen or file of the server and of MHMON is made with `printf` in any of its forms: they go through the FAO engine of StarLet, in the OpenVMS directives: `!AZ`, `!UL`, `!SL`, `!UQ`, `!XW`, `!%S` and the rest. (A path built by `snprintf` is not output.) The tests are free to write as they like.
- **The failure of a C RTL routine is reported where it happens**, with the name of the routine and `errno`:

```
if ( 0 > (a_port->sd = socket(AF_PACKET, SOCK_RAW | SOCK_NONBLOCK | SOCK_CLOEXEC, htons(NISCA$K_ETHERTY
    return  $MHMSG(MH$_SYSERR, "socket(AF_PACKET)", errno, strerror(errno));
```

Returning a condition upward and leaving the account of the run-time library behind loses the only information that would have identified the failure.

5.7 Strings and Lengths

A string is carried with its length: the counted strings of StarLet (`ASC`, `$ASCLEN`, `$ASCPTR`). A length once known is not computed again; `strlen` over the same string twice is a fault of style.

5.8 The Build Files and the Shell

- No dollar sign in an identifier of a CMake file: CMake would take it for the start of a reference. The constants of the build are named `MH_`*NAME*.
- No dollar sign in the name of an environment variable: `MSCPHUB_HELPLIB`, not `MSCPHUB$HELPLIB`.

5.9 The Documentation

- Every manual exists in English and in Russian, and the two say the same things.
- The English is plain: short sentences, common words. Many readers read it as a second language.
- Every message has a topic in both help libraries: what it means, and what to do about it.
- The manuals are laid out like the manuals of OpenVMS: a title page with the order number, a preface, numbered chapters, a command dictionary, a chapter of messages, a chapter of troubleshooting.

Chapter 6. Making a Change

6.1 Adding a Qualifier

1. Add the field to `MH$CFG` in `mh_def.h`.
2. Add a line to `s_opts` in `mscphub.c`: the name in lowercase, the field, the size, the type.
3. Check its value in `s_mh$check_cfg`; a wrong one is `MH$_IVPARAM`, saying what is expected.
4. Document it: a topic `## /NAME -- ...` in both help libraries, a paragraph in the manual page, an entry in the command dictionary of both User's Guides.

The test `docs` fails until the topics and the paragraph are there: it takes every name of `s_opts` and looks for them.

6.2 Adding a Message

1. Add the message number to the `enum` of `mh_def.h` and the condition `MH$_`*NAME* with its severity.
2. Add the record to `s_msgtab` in `mh_utl.c`: the identification, a comma, the text with FAO directives.
3. Document it: a topic *###NAME* in both help libraries, an entry in Chapter 6 of both User's Guides.

The test `docs` fails until the topics are there.

6.3 Adding an MSCP Command

The table of the commands, `s_opdef` in `mh_mscp.c`, gives each opcode the length of the command it takes (0: not taken), the length of its end message, the modifiers it may carry, and its name. Whether a command is immediate, sequential or runs side by side is decided by its opcode where the command is taken in. A command that moves data has steps in `mh_mscp.c` and a request type in `mh_bk.c`. Every new end message must pass the check that holds back an answer a node would crash on; extend the check if the new message has fields it does not know.

6.4 Adding a Counter to MHMON and Modbus

1. Add the field to `MH$STAT`, `MH$STUNIT` or `MH$STNODE` in `mh_stat.h`, and raise `MH$K_STAT_VERSION`.
2. Copy it in `mh$stat_tick` or its helpers in `mh_stat.c`.
3. For MHMON: an item or a row in `mhmon.c`, and its column in the T4 file.
4. For Modbus: a register in `s_mh$mb_layout` in `mh_mbus.c`, at a place of the map not taken; raise `MH$K_MB_MAPVERS` if a register moves.
5. Document it: the User's Guide, Section 4.8 or Appendix D, in both languages.

6.5 Releasing a Version

1. Raise `__IDENT__` and `__REV__` of the head module, and of every module changed.
2. Add a line to the modification history of every file changed.
3. Update the title blocks of the manuals: the order number, the version, the date, and what the edition supersedes.
4. `ctest` must pass.
5. `cmake --build build --target dist` makes `MSCPHUB-`*ident*`.tar.gz` of the committed sources.

Chapter 7. The Build System

7.1 Layout

Directory	Contents
`src/`	The modules
`include/`	`mh_def.h`, `nisca.h`, `mscp.h`
`doc/`	The manuals, the help library sources, the manual page, `mkpdf.py`
`etc/`	The sample parameter file, the systemd units, the scripts of OpenRC and System V init
`tests/`	The tests
`cmake/`	The uninstall script

7.2 Dependencies

Package	Use
StarLet 1.6 or later	Messages, logging, the command line, counted strings, FAO
liburing	The backend
HELP (optional)	The help library and `/HELP`
pandoc and XeLaTeX (optional)	The PDF of the manuals

7.3 Options

Option	Default	Effect
`CMAKE_INSTALL_PREFIX`	`/opt/mscphub`	Where the server is installed
`MH_STRICT`	OFF	Compiler warnings become errors
`BUILD_TESTING`	ON	The tests are built
`MH_SYSTEMD`	ON where systemd runs	The systemd units are installed (never enabled)
`MH_CONFDIR`	`/etc/mscphub` (`etc/mscphub` of another prefix)	Where the parameter file of the service is
`MH_INIT`	`OPENRC`, `SYSV` or `NONE`, by the host	The init scripts installed in `MH_INITDIR`, `/etc/init.d` (never enabled)
`MH_PDF`	OFF	The manuals are rendered as PDF

7.4 The Checks of the Configuration

CMake refuses a tree in which the `__REV__` of a module does not follow its `__IDENT__` (Section 5.4). The version of the product is read from `src/mscphub.c`.

7.5 The PDF of the Manuals

With `-DMH_PDF=ON`, `doc/mkpdf.py` renders every manual through pandoc and XeLaTeX into a PDF beside its source. The rendered manuals are kept in the repository with their sources, so that the reader does not need TeX to read them.

7.6 The Kit

`cmake --build build --target dist` makes `MSCPHUB-`*ident*`.tar.gz` by `git archive HEAD`: the committed sources, the manuals and the tests, and nothing of a build tree. What is not committed is not in the kit.

Chapter 8. Testing

8.1 What Is There

Test	What it guards
`wire`	The frames the server builds, octet by octet, against reference frames of OpenVMS V8.3 (`tests/t_frames.h`); the channel handshake
`vc`	The virtual circuit and the path: retransmission, duplicates, out of order, an old channel, 30 timeouts, HOSTSHUT, restart
`scs`	SCS and the directory; the MSCP server: STCON, GET UNIT STATUS, ONLINE, AVAILABLE, the checks of a command, credits
`blk`	READ, WRITE, COMPARE and ERASE over the block transfers; the order of the commands; ABORT; a connection that goes while the disk has a command
`bk`	The backend: io_uring, read-modify-write, ranges of sectors, the cache of written sectors, ERASE, group commit, direct I/O
`mchan`	A node of two adapters: the second channel serves the circuit of the first, a packet goes by the channel of the larger buffer with its sequence number, the circuit outlives a channel, a packet too large for the channels left fails it at once, a SOLICIT_SRV closes the other channels
`ods`	The reader of Files-11 on a small volume made by the test (`tests/t_vol.h`): the home block, INDEXF.SYS, an alias, versions, extents and extension headers, a wrong sequence number or checksum
`mop`	The MOP server: a satellite loaded from the volume of `ods`, V4 and V3, the parameters item by item, the timers, the requests not answered
`stat`	The statistics block: made, copied every 100 ms, never a torn copy under a writer in another thread, another layout refused, the queues; the answers of Modbus
`t4`	MHMON records a published block: the header of T4, the same fields in every row, the rate of a unit

Test	What it guards
`init`	The System V scripts: no start without the parameter file, the collector of T4 started, its state, stopped with its file closed
`params`	The checks of the qualifiers
`docs`	Every qualifier of the server and of MHMON and every message is documented
`helplib`	`/HELP` answers out of the library (with the HELP package)

8.2 Rules

- A change that alters what the server sends is not finished without a test that fails before it and passes after it.
- A test is checked by breaking the code it guards and seeing it fail.
- `ctest` passes before anything is installed.

Appendix A. Known Limitations

Limitation	Where it lives
8 disks per server	`/UNIT0` to `/UNIT7` in `s_opts`, `mscphub.c`
One LAN adapter	`mh_lan.c`
8 Modbus clients at a time	`mh_mbus.c`
64 nodes remembered	`mh_cc.c`, `MANYNODES`
Window of 16 packets per circuit	`mh_tr.c`
Jumbo frames up to an MTU of 9000	`MH$K_LAN_MAXMTU`, `mh_def.h`
No member disk of a shadow set (no forced error)	`mh_mscp.c`
No tapes	not implemented
8 satellites loaded by MOP	`/SATELLITE0` to `/SATELLITE7` in `s_opts`, `mscphub.c`
MOP for Alpha satellites only; no BOOTP or TFTP: an OpenVMS node loads an Integrity satellite	not implemented
No cluster over IP	not implemented